Sesión de Clase: Patrones Estructurales COMPOSITE y DECORATOR

Curso: Patrones de Diseño de Software (PIS1003)
Corte: 2 (Segundo Corte) — Semana 8
Modalidad: Presencial — Aprendizaje Basado en Proyectos (ABPr)
Docente: Eliecer Montero Ojeda
Prerrequisitos: Patrones creacionales (Singleton, Factory Method, Abstract Factory, Builder) y estructurales (Adapter, Bridge) vistos en semanas anteriores.

1.2. Conexión con lo aprendido

- **Con los creacionales:** Los patrones creacionales resuelven *cómo crear* objetos (Singleton garantiza instancia única; Factory Method delega la creación; Abstract Factory crea familias; Builder construye paso a paso). **Composite y Decorator resuelven *cómo componer* esos objetos ya creados** en estructuras más ricas.
- **Con Adapter y Bridge (S7):** Adapter traduce interfaces; Bridge separa dos jerarquías ortogonales. **Composite** también organiza jerarquías, pero de forma **recursiva** (un objeto contiene objetos del mismo tipo). **Decorator** también envuelve objetos como Adapter, pero su intención es **añadir responsabilidades**, no traducir interfaces.

---

2. CONTENIDO TEÓRICO

BLOQUE 1 — Introducción: los patrones estructurales y su lugar en el catálogo GoF

1.1. Recordatorio: ¿dónde estamos?

Los patrones estructurales responden a la pregunta: **¿cómo compongo objetos y clases para formar estructuras más grandes manteniendo flexibilidad y eficiencia?**

Hasta ahora hemos visto:

Categoría	Patrón	Pregunta que responde
Creacional	Singleton	¿Cómo garantizo una única instancia?
Creacional	Factory Method	¿Cómo delego la creación a subclases?
Creacional	Abstract Factory	¿Cómo creo familias de objetos relacionados?
Creacional	Builder	¿Cómo construyo objetos complejos paso a paso?
Estructural	**Adapter**	¿Cómo hago compatibles interfaces incompatibles?
Estructural	**Bridge**	¿Cómo desacoplo abstracción de implementación?
Estructural	**Composite**	¿Cómo trato objetos individuales y composiciones de forma uniforme?
Estructural	**Decorator**	¿Cómo añado responsabilidades dinámicamente sin herencia?

1.2. Los dos problemas que resolveremos hoy

Problema A — Jerarquías recursivas: En el Sistema de Gestión Académica, una Facultad contiene Programas, un Programa contiene Cursos, un Curso contiene Estudiantes. Pero también hay "grupos de cursos" (un paquete de materias optativas) y "estudiantes individuales". ¿Cómo calcular el promedio académico de una Facultad entera sin escribir código distinto para cada nivel?

Problema B — Extensiones dinámicas: Un estudiante puede tener beca, descuento por hermanos, descuento por pronto pago, o una combinación de los tres. Además, el cálculo de la matrícula puede variar en runtime. ¿Cómo añadimos funcionalidades sin crear 2^n combinaciones de clases?

Composite resuelve el Problema A. **Decorator** resuelve el Problema B.

BLOQUE 2 — Patrón COMPOSITE

2.1. Propósito

Componer objetos en estructuras de árbol para representar jerarquías parte-todo, permitiendo que los clientes traten de manera uniforme a objetos individuales (hojas) y a composiciones (ramas).

Intención GoF: *"Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly."*

2.2. Motivación: la jerarquía académica

En el proyecto integrador necesitamos calcular métricas agregadas sobre la estructura académica:

Facultad de Ingenierías

├── Programa de Ingeniería de Sistemas

│ ├── Curso de Patrones de Diseño

│ │ ├── Estudiante Ana

│ │ └── Estudiante Luis

│ └── Curso de Bases de Datos

│ ├── Estudiante Ana

│ └── Estudiante Pedro

└── Programa de Ingeniería Industrial

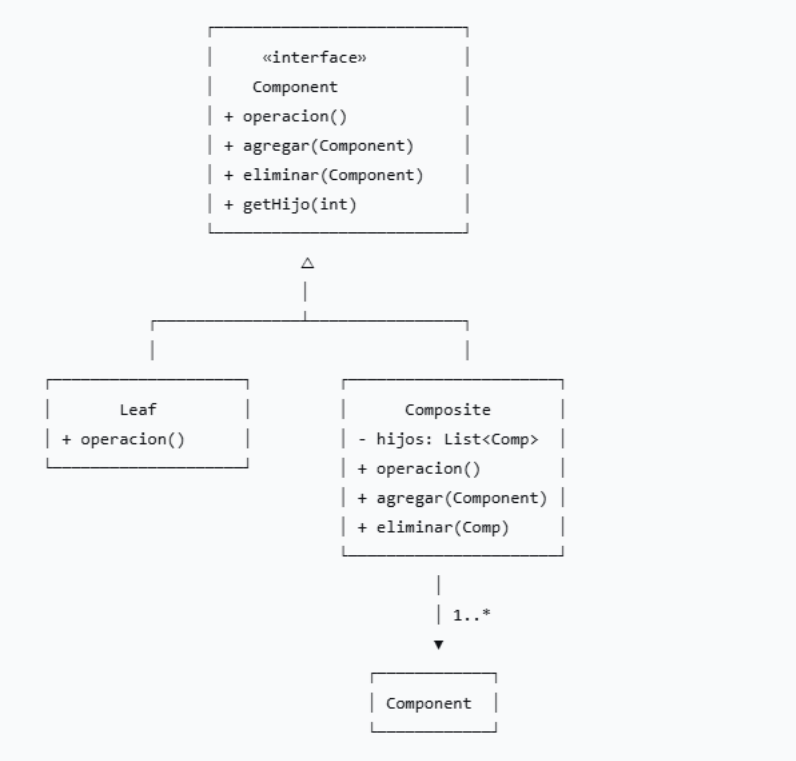
└── Curso de Procesos

└── Estudiante Carla

Pregunta: ¿Cómo calcular el promedio académico de toda la Facultad sin escribir if (esFacultad) ... else if (esPrograma) ... else if (esCurso) ...?

Respuesta Composite: definimos una interfaz común ElementoAcademico que declara getPromedio(), y tanto las hojas (Estudiante) como las ramas (Curso, Programa, Facultad) la implementan. Las ramas delegan recursivamente en sus hijos.

2.3. Estructura (participantes)



Participante	Rol
Component	Interfaz común para hojas y composiciones
Leaf	Objeto individual (no tiene hijos)
Composite	Contiene hijos (que pueden ser hojas u otros composites); delega operaciones
Client	Manipula objetos a través de la interfaz Component sin distinguir

2.4. Implementación en el proyecto Sistema General Académico

Component:

```
package co.uts.academico.dominio;

public interface ElementoAcademico {
    double getPromedio();
    int getTotalEstudiantes();
    void mostrar(String indentacion);
}
```

Leaf – Estudiante:

```
package co.uts.academico.dominio;

public class Estudiante implements ElementoAcademico {
    private final String nombre;
    private final double promedio;

    public Estudiante(String nombre, double promedio) {
        this.nombre = nombre;
        this.promedio = promedio;
    }

    @Override public double getPromedio() { return promedio; }
    @Override public int getTotalEstudiantes() { return 1; }
    @Override public void mostrar(String ind) {
        System.out.printf("%s- Estudiante: %s (promedio %.2f)%n", ind, nombre, promedio);
    }
}
```

Composite — Curso, Programa, Facultad:

```java
package co.uts.academico.dominio;

import java.util.ArrayList;
import java.util.List;

public class Curso implements ElementoAcademico {
    private final String nombre;
    private final List<ElementoAcademico> estudiantes = new ArrayList<>();

    public Curso(String nombre) { this.nombre = nombre; }

    public void agregar(ElementoAcademico e) { estudiantes.add(e); }
    public void eliminar(ElementoAcademico e) { estudiantes.remove(e); }

    @Override
    public double getPromedio() {
        return estudiantes.stream()
                .mapToDouble(ElementoAcademico::getPromedio)
                .average().orElse(0.0);
    }

    @Override
    public int getTotalEstudiantes() {
        return estudiantes.stream()
                .mapToInt(ElementoAcademico::getTotalEstudiantes)
                .sum();
    }

    @Override
    public void mostrar(String ind) {
        System.out.println(ind + "+ Curso: " + nombre);
        estudiantes.forEach(e -> e.mostrar(ind + "  "));
    }
}
```

Cliente uniforme:

```java
public class ReporteAcademico {
    public static void main(String[] args) {
        Estudiante ana  = new Estudiante("Ana", 4.5);
        Estudiante luis = new Estudiante("Luis", 3.8);
        Estudiante pedro = new Estudiante("Pedro", 4.1);

        Curso patrones = new Curso("Patrones de Diseño");
        patrones.agregar(ana);
        patrones.agregar(luis);

        Curso bd = new Curso("Bases de Datos");
        bd.agregar(ana);
        bd.agregar(pedro);

        Programa sistemas = new Programa("Ing. Sistemas");
        sistemas.agregar(patrones);
        sistemas.agregar(bd);

        Facultad ingenierias = new Facultad("Ingenierías");
        ingenierias.agregar(sistemas);

        // El cliente trata igual a un estudiante y a una facultad
        System.out.println("Promedio facultad: " + ingenierias.getPromedio());
        System.out.println("Total estudiantes: " + ingenierias.getTotalEstudiantes());
        ingenierias.mostrar("");
    }
}
```

Nota: Programa y Facultad son composites idénticos a Curso en estructura (solo cambia la semántica del nombre). En un diseño minimalista podrían reutilizar la misma clase CompositeAcademico.

2.5. Consecuencias

Ventajas:

- **Uniformidad:** el cliente trata hojas y composites por igual.
- **Recursividad natural:** las operaciones se propagan por el árbol sin if/else por tipo.
- **Extensibilidad:** agregar nuevos tipos de nodos (ej. GrupoOptativo) no modifica al cliente.
- **Open/Closed:** se pueden añadir nuevos componentes sin tocar el código cliente.

Desventajas / trade-offs:

- **Sobre-generalización:** el diseño puede volverse demasiado abstracto si la jerarquía es simple.
- **Dificulta restricciones:** si un composite solo debe contener ciertos tipos de hijos, el patrón no lo impone directamente.
- **Riesgo de ciclos:** si se permite agregar un composite dentro de sí mismo, se debe validar.
- **Operaciones no uniformes:** si algunas operaciones solo aplican a hojas o solo a composites, se contamina la interfaz Component.

2.6. Cuándo usar Composite

- Cuando la estructura del dominio es **naturalmente jerárquica** (árboles).
- Cuando el cliente debe **ignorar la diferencia** entre hojas y ramas.
- Cuando las operaciones se pueden definir recursivamente.
- **No lo uses** si la jerarquía tiene profundidad fija y conocida (ahí basta con clases concretas).

2.7. Ejemplos reales

- **Sistema de archivos:** File y Directory comparten FileSystemNode.
- **Interfaz gráfica:** Component de Swing, Panel que contiene otros Component.
- **DOM HTML:** un div contiene otros elementos.
- **Expresiones aritméticas:** número + suma + multiplicación.

2.8. Conexión con el proyecto integrador

En el Sistema de Gestión Académica, **Composite** permite:

- Calcular el promedio de una Facultad, Programa o Curso con el mismo método.
- Generar reportes jerárquicos (Facultad → Programa → Curso → Estudiante).
- Aplicar operaciones masivas (ej. matrícula masiva, cierre de semestre) sin recorrer manualmente cada nivel.
- Extender la estructura con nuevos nodos (ej. "Área de formación") sin tocar el cliente.

BLOQUE 3 — Patrón DECORATOR

3.1. Propósito

Añadir responsabilidades a un objeto dinámicamente, envolviéndolo en uno o más objetos decoradores que comparten su interfaz. Decorator ofrece una alternativa flexible a la herencia para extender funcionalidad.

Intención GoF: *"Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."*

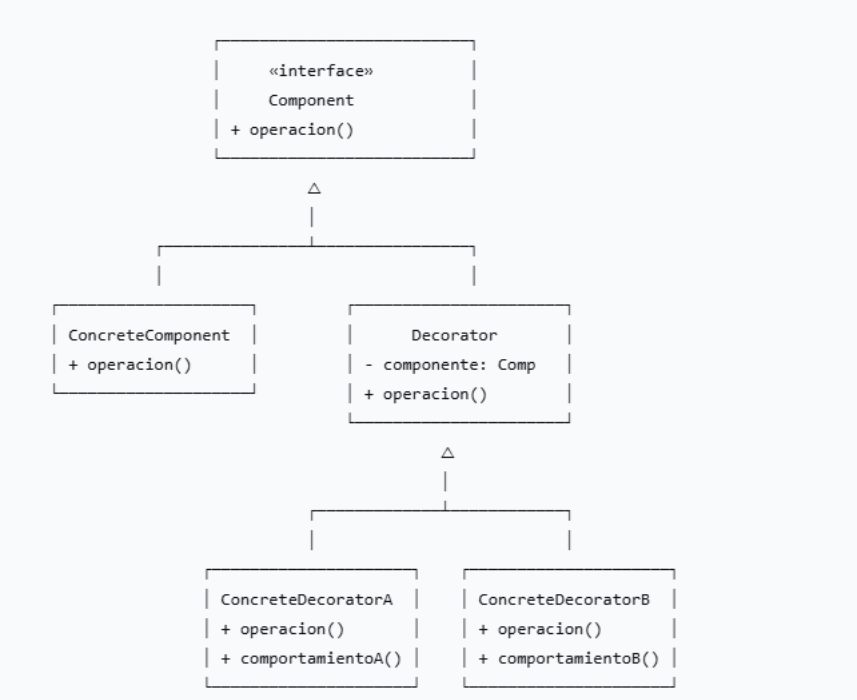
3.2. Motivación: el cálculo de la matrícula

En el proyecto integrador, el costo de matrícula de un estudiante puede verse afectado por múltiples factores combinables:

- Beca por mérito académico (-30%).
- Descuento por hermanos (-10%).
- Descuento por pronto pago (-5%).
- Recargo por curso intersemestral (+15%).
- Recargo por material adicional (+$50.000).

Un estudiante puede tener **cualquier combinación** de estos. Modelar con herencia significaría crear 2^5 = 32 clases (EstudianteConBecaYHermanosYProntoPago...). **Decorator** resuelve esto permitiendo envolver dinámicamente el cálculo base.

3.3. Estructura (participantes)



Participante	Rol
Component	Interfaz común; define las operaciones que pueden decorarse
ConcreteComponent	Objeto base al que se añaden responsabilidades
Decorator	Mantiene referencia a un Component y delega en él
ConcreteDecorator	Añade comportamiento antes/después de delegar

3.4. Implementación en el proyecto integrador

Component — cálculo de matrícula:

```
package co.uts.academico.matricula;

public interface CalculoMatricula {
    double calcular();
    String detalle();
}
```

ConcreteComponent — costo base:

```
package co.uts.academico.matricula;

public class MatriculaBase implements CalculoMatricula {
    private final double costoBase;

    public MatriculaBase(double costoBase) { this.costoBase = costoBase; }

    @Override public double calcular() { return costoBase; }
    @Override public String detalle() {
        return String.format("Matrícula base: $%.2f%n", costoBase);
    }
}
```

Decorator abstracto:

```
package co.uts.academico.matricula;

public abstract class MatriculaDecorator implements CalculoMatricula {
    protected final CalculoMatricula componente;

    protected MatriculaDecorator(CalculoMatricula componente) {
        this.componente = componente;
    }

    @Override public double calcular() { return componente.calcular(); }
    @Override public String detalle() { return componente.detalle(); }
}
```

ConcreteDecorators:

```
package co.uts.academico.matricula;

public class BecaMerito extends MatriculaDecorator {
    private static final double PORCENTAJE = 0.30;

    public BecaMerito(CalculoMatricula c) { super(c); }

    @Override public double calcular() {
        return componente.calcular() * (1 - PORCENTAJE);
    }

    @Override public String detalle() {
        return componente.detalle() +
               String.format("  - Beca mérito (-%.0f%%): -$%.2f%n",
                   PORCENTAJE * 100,
                   componente.calcular() * PORCENTAJE);
    }
}

public class DescuentoHermanos extends MatriculaDecorator {
    private static final double PORCENTAJE = 0.10;
    public DescuentoHermanos(CalculoMatricula c) { super(c); }
    @Override public double calcular() { return componente.calcular() * (1 - PORCENTAJE
    @Override public String detalle() {
        return componente.detalle() +
               String.format("  - Descuento hermanos (-%.0f%%): -$%.2f%n",
                   PORCENTAJE * 100, componente.calcular() * PORCENTAJE);
    }
}

public class RecargoIntersemestral extends MatriculaDecorator {
    private static final double PORCENTAJE = 0.15;
    public RecargoIntersemestral(CalculoMatricula c) { super(c); }
    @Override public double calcular() { return componente.calcular() * (1 + PORCENTAJE
    @Override public String detalle() {
        return componente.detalle() +
               String.format("  + Recargo intersemestral (+%.0f%%): +$%.2f%n",
                   PORCENTAJE * 100, componente.calcular() * PORCENTAJE);
    }
}
```

Cliente — composición dinámica:

```java
public class LiquidacionMatricula {
    public static void main(String[] args) {
        // Estudiante con beca + descuento por hermanos
        CalculoMatricula c1 = new MatriculaBase(5_000_000);
        c1 = new BecaMerito(c1);
        c1 = new DescuentoHermanos(c1);
        System.out.println(c1.detalle());
        System.out.printf("TOTAL: $%.2f%n%n", c1.calcular());

        // Estudiante con beca + recargo intersemestral
        CalculoMatricula c2 = new MatriculaBase(5_000_000);
        c2 = new BecaMerito(c2);
        c2 = new RecargoIntersemestral(c2);
        System.out.println(c2.detalle());
        System.out.printf("TOTAL: $%.2f%n", c2.calcular());
    }
}
```

3.5. Consecuencias

Ventajas:

- **Más flexible que la herencia:** las responsabilidades se añaden/eliminan en runtime.
- **Evita explosión combinatoria:** en lugar de 2^n subclases, se tienen n decoradores combinables.
- **Open/Closed:** nuevos decoradores se agregan sin tocar código existente.
- **Responsabilidades pequeñas y enfocadas:** cada decorador hace una sola cosa.
- **Composición en cualquier orden:** los decoradores se pueden anidar libremente.

Desventajas / trade-offs:

- **Muchos objetos pequeños:** puede dificultar el debugging y la trazabilidad.
- **Identidad del objeto:** un decorador no es idéntico al componente decorado (equals puede fallar).
- **Orden importa:** en algunos casos el orden de decoración altera el resultado (ej. aplicar porcentajes vs. montos fijos).
- **Complejidad de la interfaz:** si Component tiene muchos métodos, cada decorador debe delegarlos todos.

3.6. Cuándo usar Decorator

- Cuando quieres añadir responsabilidades a objetos **individuales** (no a toda la clase).
- Cuando las responsabilidades pueden **combinarse** de formas diversas.
- Cuando la herencia produciría **explosión combinatoria**.
- Cuando necesitas añadir/quitar comportamiento en **runtime**.
- **No lo uses** si las responsabilidades son fijas y conocidas en tiempo de diseño.

3.7. Ejemplos reales

- java.io**:** BufferedReader(new FileReader(...)), DataInputStream(new BufferedInputStream(...)).
- **Java Servlet:** HttpServletRequestWrapper para añadir comportamiento a requests.
- **UI frameworks:** bordes, sombras, scroll añadidos a componentes.
- **Spring Security:** filtros que envuelven la cadena de seguridad.

3.8. Conexión con el proyecto integrador

En el Sistema de Gestión Académica, **Decorator** permite:

- Calcular la matrícula con cualquier combinación de becas, descuentos y recargos.
- Añadir bonificaciones al cálculo de notas (participación, trabajos extra, curva).
- Decorar reportes con encabezados, pies de página, marcas de agua.
- Aplicar validaciones adicionales a inscripciones sin modificar el flujo base.

BLOQUE 4 — Comparativa Composite vs. Decorator y relación con Adapter/Bridge

4.1. Tabla comparativa

Criterio	**Composite**	**Decorator**
Intención	Tratar uniformemente hojas y composiciones	Añadir responsabilidades dinámicamente
Estructura	Árbol (1..* hijos)	Cadena/cadena de envoltura (1 componente)
Número de hijos	Muchos (composición)	Uno (envoltura)
Relación con hijos	"Tiene muchos" (agregación)	"Envuelve a uno" (composición)
Objetivo principal	Uniformidad de operaciones	Extensión de comportamiento
¿Cambia la interfaz?	No, la conserva	No, la conserva
Alternativa a	Herencia con if/else por tipo	Explosión combinatoria de subclases
Cliente ve	Un árbol transparente	Un objeto envuelto transparente
Ejemplo en proyecto	Facultad → Programa → Curso → Estudiante	Matrícula con becas + descuentos

4.2. Relación con Adapter y Bridge

Patrón	¿Envuelve?	¿Cuántos?	¿Cambia interfaz?	¿Añade comportamiento?
Adapter	Sí	1 (Adaptee)	**Sí** (traduce)	No (solo traduce)
Bridge	Sí (referencia)	1 (Implementor)	No	No (delega)
Composite	Sí (contiene)	Muchos	No	No (agrega)
Decorator	Sí	1	No	**Sí**

Regla mnemotécnica:

- **Adapter** = traductor (cambia interfaz).
- **Bridge** = arquitecto (separa jerarquías).
- **Composite** = árbol (trata hojas y ramas igual).
- **Decorator** = capas (añade sin cambiar).

4.3. Combinación de patrones en el proyecto integrador

Un diseño maduro del Sistema de Gestión Académica puede combinar los cuatro patrones:

- **Composite** para la estructura Facultad → Programa → Curso → Estudiante.
- **Decorator** para el cálculo de matrícula con becas y descuentos.
- **Adapter** para integrar pasarelas de pago heterogéneas.
- **Bridge** para notificaciones multicanal.

Esto evidencia cómo los patrones se complementan, no compiten.

BLOQUE 5 — Cierre, preguntas de reflexión y conexión con el proyecto

5.1. Preguntas para la reflexión

1. ¿Por qué Composite es útil cuando la jerarquía es natural y Decorator cuando la extensión es dinámica?
2. ¿En qué casos preferirías herencia sobre Composite o Decorator? (Cuando la variación es fija y pequeña).
3. ¿Qué pasaría si en el cálculo de matrícula aplicaras primero RecargoIntersemestral y luego BecaMerito vs. al revés? (El resultado puede diferir).
4. ¿Puede un Composite también ser Decorator? (Sí, aunque es inusual; el decorador envuelve un solo componente, el composite contiene muchos).
5. ¿Qué relación tienen ambos patrones con los principios SOLID? (Open/Closed, composición sobre herencia, responsabilidad única).

5.2. Recursos para profundizar

Bibliografía básica:

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Composite: pp. 163–173; Decorator: pp. 175–184).
- Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly. (Cap. 3: Decorator; Cap. 9: Composite).
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. (Cap. 8–12: SOLID).

Webgrafía:

- Refactoring.Guru. (2023). *Composite*. https://refactoring.guru/design-patterns/composite
- Refactoring.Guru. (2023). *Decorator*. https://refactoring.guru/design-patterns/decorator
- SourceMaking. (2023). *Composite Pattern*. https://sourcemaking.com/design_patterns/composite

- SourceMaking. (2023). *Decorator Pattern*. https://sourcemaking.com/design_patterns/decorator

Software y herramientas:

- Java 17 + IntelliJ IDEA/Eclipse.
- PlantUML para diagramas de clases.
- Git/GitHub para control de versiones.
- JUnit 5 para pruebas.

5.3. Errores comunes

Error	Corrección
Confundir Composite con "lista de objetos"	Composite requiere interfaz común y tratamiento uniforme
Usar Decorator para cambiar interfaz	Decorator **no cambia** la interfaz; para eso está Adapter
No delegar en el componente decorado	Cada decorador debe llamar a componente.operacion()
Olvidar el orden de decoración	Documentar el orden cuando el resultado depende de él
Crear decoradores que hacen demasiado	Cada decorador debe tener **una** responsabilidad
No escribir pruebas para las combinaciones	Probar base, +1, +2 en distinto orden, +3
No documentar el ADR	El ADR es parte de la evidencia del corte

6. CIERRE DE LA SESIÓN

6.1. Ideas clave para llevarse

1. **Composite** modela jerarquías parte-todo y permite tratar hojas y ramas por igual.
2. **Decorator** añade responsabilidades dinámicamente sin explosión combinatoria de subclases.
3. Ambos favorecen **composición sobre herencia** y cumplen **Open/Closed**.
4. Ambos **conservan la interfaz** del componente (a diferencia de Adapter, que la traduce).
5. En el proyecto integrador: **Composite** para Facultad → Programa → Curso → Estudiante; **Decorator** para matrícula con becas y descuentos.
6. Los cuatro patrones estructurales vistos (Adapter, Bridge, Composite, Decorator) se **complementan** en un diseño maduro.